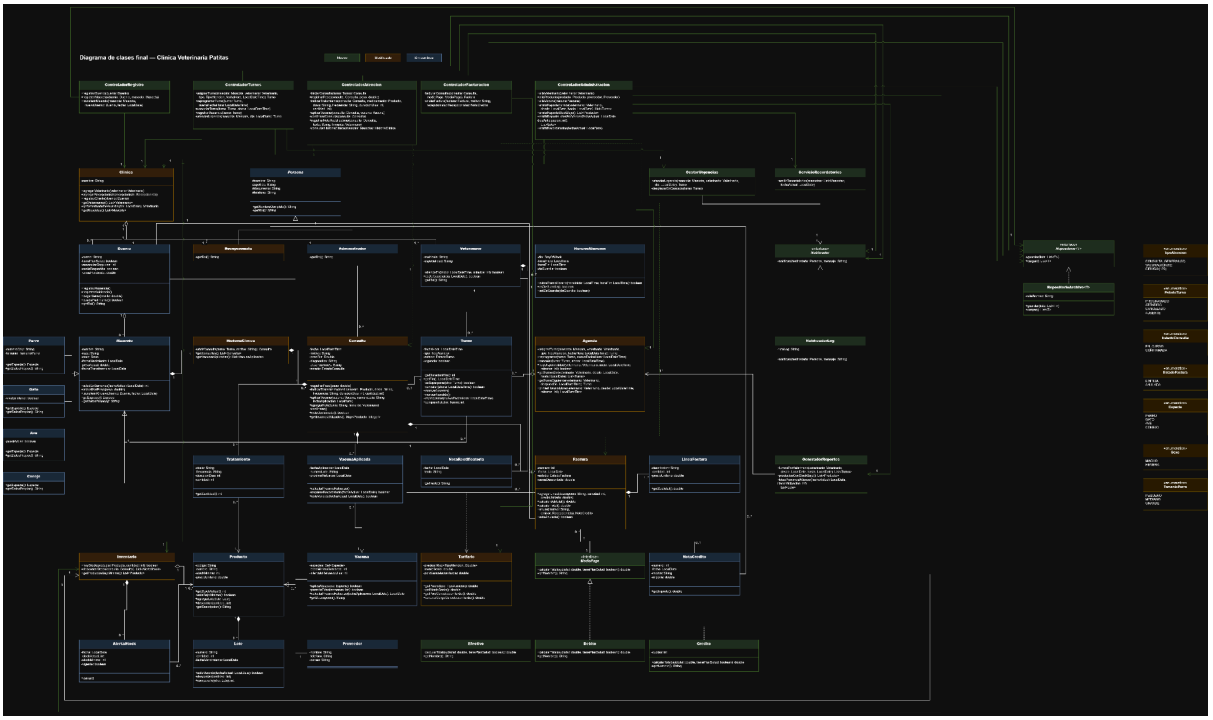
Punto 6 - Diagrama de clases final



Convenciones

- **Visibilidad:** + público, - privado, # protegido.
- **Clase abstracta o interfaz:** el nombre va en cursiva; las interfaces llevan ««interface»».
- **Composición (rombo lleno):** la parte no existe sin el todo. **Agregación (rombo hueco):** el todo agrupa partes que existen por sí solas.
- **Asociación navegable (flecha abierta):** la clase de origen guarda una referencia a la de destino. **Dependencia (flecha punteada):** la usa de forma pasajera, por ejemplo como parámetro.
- **Flechas verdes:** relaciones de los controladores y servicios con las clases que coordinan.
- Los atributos que son colecciones o referencias a otras clases del modelo no se repiten como atributos: quedan implícitos en la relación y su multiplicidad.
- Las enumeraciones se muestran aparte y se usan como tipos de atributos.

1. Qué cambió respecto del diagrama inicial

- **Controladores (nuevos):** ControladorRegistro, ControladorTurnos, ControladorAtencion, ControladorFacturacion y ControladorAdministracion. El «Sistema» de los diagramas de secuencia pasa a ser el controlador correspondiente.
- **Agenda se dividió:** las urgencias pasaron a GestorUrgencias y Agenda dejó de notificar.

- **MedioPago:** dejó de ser un enum y pasó a ser una interfaz con Efectivo, Debito y Credito.
- **Notificaciones:** Notificador (interfaz) y NotificadorLog reemplazan a Agenda.notificarDuenio e Inventario.notificarAdministrador. Inventario.descontarStock devuelve las alertas generadas para que el controlador las notifique.
- **Persistencia:** Repositorio<T> (interfaz) y RepositorioArchivo<T> reemplazan a Clinica.guardarEnArchivo y cargarDesdeArchivo.
- **Reportes y recordatorios:** GeneradorReportes (turnos por veterinario, stock bajo y lotes próximos a vencer) y ServicioRecordatorios.
- **Anulación de facturas:** Recepcionista.emitirNotaCredito desapareció; Factura.anular(motivo, emisor) genera la NotaCredito.
- **Cliente de la factura:** Factura se asocia con Duenio y calcularTotal() ya no recibe el plan de salud.
- **Creación de las partes:** HistoriaClinica.abrirConsulta, Consulta.indicarTratamiento, aplicarVacuna y agregarNota, y Factura.agregarLinea crean ellas mismas las partes de sus composiciones.
- **Otros ajustes:** Clinica suma getVeterinarioDeGuardia y getMascotas; Tarifario suma calcularCargoCancelacionTardia; Inventario.descontarStock devuelve las alertas generadas.

2. Controladores (GRASP: Controlador)

Cada controlador es el punto de entrada de una familia de casos de uso. No contiene reglas de negocio: recibe la solicitud de la interfaz, busca los objetos necesarios, delega en ellos y, si hace falta, notifica o persiste. No tienen atributos propios; las clases que coordinan son las asociaciones del diagrama.

ControladorRegistro

Casos de uso de alta de dueños y mascotas (CU-01, CU-02, CU-03).

- **Métodos:**
  - registrarDuenio(duenio): da de alta al cliente en la clínica y lo guarda.
  - registrarMascota(duenio, mascota): asocia la mascota al dueño.
  - transferirMascota(mascota, nuevoDuenio, fecha): delega en Mascota.transferirA; la historia clínica queda intacta.
- **Relaciones:** asociación navegable con Clinica y con Repositorio.

ControladorTurnos

Casos de uso de turnos y urgencias (CU-04 a CU-08).

- **Métodos:**
  - asignarTurno(mascota, veterinario, tipo, fechaHora): Turno: verifica que el dueño pueda pedir turno y delega en Agenda.
  - reprogramarTurno(turno, nuevaFechaHora) y cancelarTurno(turno, ahora): delegan en Agenda; al reprogramar, notifica al dueño.

    - registrarAusencia(turno): marca el turno como ausente y suma la ausencia al dueño (RN-T4).
    - atenderUrgencia(mascota, dia): Turno: obtiene el veterinario de guardia de Clinica y delega en GestorUrgencias.
- **Relaciones:** asociación navegable con Agenda, GestorUrgencias, Clinica, Notificador y Repositorio.

ControladorAtencion

Casos de uso de la atención clínica (CU-16 a CU-25).

- **Métodos:**
    - iniciarConsulta(turno): Consulta: abre la consulta en la historia clínica de la mascota.
    - registrarPeso, indicarTratamiento, aplicarVacuna y registrarNotaRectificatoria: reciben la consulta y los datos sueltos y delegan en Consulta.
    - confirmarConsulta(consulta): confirma la consulta, hace que Inventario descuente el stock y notifica al administrador las alertas de stock bajo que devuelva.
    - consultarHistoriaClinica(mascota): HistoriaClinica: devuelve la historia para mostrarla.
- **Relaciones:** dependencia con Consulta (la recibe por parámetro); asociación navegable con Inventario y Notificador.

ControladorFacturacion

Casos de uso de CU-09 y CU-10.

- **Métodos:**
    - facturarConsulta(consulta, medioPago): Factura: crea la factura de una consulta confirmada, con la línea de la consulta (precio del Tarifario) y una línea por insumo, y la guarda.
    - anularFactura(factura, motivo, recepcionista): NotaCredito: delega en Factura.anular.
- **Relaciones:** dependencia con Factura; asociación navegable con Tarifario y Repositorio.

ControladorAdministracion

Casos de uso del administrador (CU-26 a CU-31) y emisión de recordatorios (CU-20).

- **Métodos:**
    - altaVeterinario, altaProducto y altaVacuna: altas de personal y de catálogos, con persistencia.
    - emitirReporteTurnos, emitirReporteStockBajo y emitirReporteLotesPorVencer(fechaActual, diasAnticipacion): delegan en GeneradorReportes.
    - emitirRecordatorios(fechaActual): pasa las mascotas de Clinica a ServicioRecordatorios.

- **Relaciones:** asociación navegable con Clinica, Inventario, GeneradorReportes, ServicioRecordatorios y Repositorio.

3. Clínica y personas

Clinica

Representa a la clínica «Patitas» y agrupa a su personal y a sus clientes.

- **Atributos:** -nombre: String.
- **Métodos:**
    - agregarVeterinario, agregarRecepcionista y registrarCliente: dan de alta cada elemento.
    - getVeterinarios(): List<Veterinario>: lista del personal profesional.
    - getVeterinarioDeGuardia(dia): Veterinario: recorre sus veterinarios y devuelve el que está de guardia ese día (S-07); lo usa ControladorTurnos en las urgencias.
    - getMascotas(): List<Mascota>: reúne las mascotas de todos sus clientes, para los recordatorios.
    - No guarda ni carga archivos: de eso se encarga el repositorio (sección 8).
- **Relaciones (agregaciones, Clinica en el extremo del rombo):** 1 a 1..* con Veterinario, 1 a 1..* con Recepcionista, 1 a 1 con Administrador y 1 a 0..* con Duenio.

Persona (abstracta)

Generaliza los datos comunes de todas las personas del sistema.

- **Atributos (protegidos):** #nombre, #apellido, #documento, #telefono.
- **Métodos:** getNombreCompleto(): String y getRol(): String (abstracto; cada subclase lo sobreescribe).
- **Relaciones:** superclase de Duenio, Veterinario, Recepcionista y Administrador. También es el tipo del destinatario de Notificador.notificar.

Duenio

Cliente de la clínica, responsable de una o más mascotas. No opera el sistema (S-01).

- **Atributos:** -correo, -tienePlanSalud (15 % de descuento, RN-F3), -ausenciasSeguidas (vuelve a cero al asistir, S-05), -seniaRequerida (RN-T4) y -seniaPendiente (monto a descontar de la próxima factura, S-04).
- **Métodos:** registrarAusencia(), registrarAsistencia(), pagarSenia(monto), puedePedirTurno(): boolean (false mientras la seña requerida no se pague) y getRol().
- **Relaciones:** hereda de Persona; agregación 1 a 0..* con Mascota; es el cliente de las Factura (0..* a 1); parte de Clinica.

Recepcionista

Personal que registra dueños y mascotas, gestiona turnos y factura. No puede registrar diagnósticos ni tratamientos (RN-M3).

- **Atributos:** ninguno propio.
- **Métodos:** getRol(). La anulación de facturas la coordina ControladorFacturacion y la ejecuta Factura.
- **Relaciones:** hereda de Persona; asociación con NotaCredito (una recepcionista emite 0..* notas); parte de Clinica.

Administrador

Gestiona veterinarios, horarios, catálogos y reportes, y recibe las alertas de stock bajo.

- **Atributos:** ninguno propio. **Métodos:** getRol().
- **Relaciones:** hereda de Persona; parte de Clinica.

Veterinario

Profesional que atiende, registra consultas, diagnósticos, tratamientos y vacunas, y firma notas rectificatorias (RN-M3, RN-M4).

- **Atributos:** -matricula, -especialidad. Sus horarios y guardias están en la relación con HorarioAtencion.
- **Métodos:** atiendeEn(inicio, minutos): boolean (busca el HorarioAtencion del día y le pregunta si cubre el tramo, RN-T2), esDeGuardia(dia): boolean y getRol().
- **Relaciones:** hereda de Persona; composición 1 a 0..7 con HorarioAtencion; asociación con Turno, Consulta y NotaRectificatoria; parte de Clinica.

HorarioAtencion

Bloque de atención de un veterinario para un día de la semana (máximo uno por día, S-22).

- **Atributos:** -dia: DayOfWeek, -horaInicio, -horaFin (LocalTime) y -deGuardia: boolean.
- **Métodos:** cubreTramoHorario(horaInicio, horaFin): boolean (el tramo entra completo en el bloque), esDeGuardia() y setDeGuardia(deGuardia).
- **Relaciones:** composición con Veterinario.

4. Mascotas e historia clínica

Mascota (abstracta), Perro, Gato, Ave y Conejo

Paciente de la clínica. Cada especie agrega datos propios (RN-M2).

- **Mascota:** -nombre, -raza, -sexo: Sexo, -fechaNacimiento, -pesoActual, -fechaTransferencia. Métodos: edadEnSemanas(fechaActual) (S-14), actualizarPeso(peso) (S-23),

transferirA(nuevoDuenio, fecha) (RN-M1, S-24) y los abstractos getEspecie() y getDatosPropios().
- **Subclases:** Perro (-numeroChip, -tamanio: TamanioPerro), Gato (-viveEnInterior), Ave (-puedeVolar) y Conejo (sin extras, S-12). Todas sobreescriben getEspecie() y getDatosPropios().
- **Relaciones:** asociada a un Duenio (el actual) y a 0..* Turno; composición 1 a 1 con HistoriaClinica.

HistoriaClinica

Conjunto de consultas de una mascota; solo crece (RN-M4).

- **Atributos:** ninguno propio.
- **Métodos:**
  - abrirConsulta(turno, motivo): Consulta: **crea** la consulta y la incorpora a la historia (GRASP: Creador).
  - getConsultas(): lista de solo lectura, en orden cronológico (CU-25).
  - getVacunasAplicadas(): reúne las vacunas de todas las consultas.
- **Relaciones:** composición con Mascota y composición 1 a 0..* con Consulta.

Consulta

Registro de una atención, con estados en curso y confirmada (S-17); al confirmarse queda inmodificable (S-18).

- **Atributos:** -fecha, -motivo, -diagnostico, -observaciones, -pesoDia: Double y -estado: EstadoConsulta.
- **Métodos:**
  - registrarPeso(peso): guarda el peso del día; sin él no se pueden indicar medicamentos (RN-M5).
  - indicarTratamiento(medicamento, dosis, frecuencia, duracionDias, cantidad): verifica el peso y **crea** el Tratamiento (Creador).
  - aplicarVacuna(vacuna, numeroLote, fechaAplicacion): **crea** la VacunaAplicada y calcula su próximo refuerzo.
  - agregarNota(texto, firmante): **crea** la NotaRectificatoria.
  - confirmar(), estaConfirmada() y getInsumosUtilizados(): Map<Producto, Integer> (insumos con cantidad, tanto para descontar stock como para armar las líneas de la factura).
- **Relaciones:** composición con HistoriaClinica; composición 1 a 0..* con Tratamiento, VacunaAplicada y NotaRectificatoria; asociación con Veterinario y con Turno (0..1 a 1); Factura la factura (0..1 a 1).

Tratamiento, VacunaAplicada y NotaRectificatoria

Son partes de Consulta; las crea la propia consulta.

- **Tratamiento:** -dosis, -frecuencia, -duracionDias, -cantidad; método getCantidad(). Asociación navegable hacia Producto (el medicamento).
- **VacunaAplicada:** -fechaAplicacion, -numeroLote, -proximoRefuerzo; métodos calcularProximoRefuerzo(), requiereRecordatorio(fechaActual) (desde 7 días antes, RN-V2) y estaVencida(fechaActual) (30 días sin refuerzo, S-26). Asociación navegable hacia Vacuna.
- **NotaRectificatoria:** -fecha, -texto; método getTexto(). Asociación 0..* a 1 con Veterinario (quien la firma, S-21).

5. Turnos y agenda

Turno

Reserva de una mascota con un veterinario, para un tipo de atención.

- **Atributos:** -fechaHora, -tipo: TipoAtencion (define la duración, RN-T1), -estado: EstadoTurno, -urgencia.
- **Métodos:** getDuracionMin(), getFin(), seSuperpone(otro) (RN-T2), cancelar(ahora): boolean (true si hay cargo por cancelación tardía, RN-T3), marcarAusente(), marcarAtendido(), reprogramar(nuevaFechaHora) y compareTo(otro) (orden por fecha y hora).
- **Relaciones:** agregado por Agenda (1 a 0..*); asociación con Mascota y Veterinario; 0..1 Consulta por turno.

Agenda

Administra los turnos de la clínica y valida su asignación. No atiende urgencias ni notifica: eso lo hacen GestorUrgencias y Notificador.

- **Atributos:** ninguno propio; los turnos son la relación.
- **Métodos:** asignarTurno(mascota, veterinario, tipo, fechaHora): Turno (**crea** el Turno, CU-04), reprogramar(turno, nuevaFechaHora) (CU-05), cancelar(turno, ahora) (CU-06), hayDisponibilidad(veterinario, inicio, minutos) (CU-11), getTurnosDe(veterinario, desde, hasta), y getTurnoSiguiente(veterinario, despuesDe) y primerHorarioLibre(veterinario, desde, minutos), que usa GestorUrgencias.
- **Relaciones:** agregación 1 a 0..* con Turno.

GestorUrgencias

Resuelve la atención de urgencias (CU-08, RN-T5, S-08, S-09). Es una responsabilidad propia, distinta de la gestión de turnos (SOLID: S; GRASP: Fabricación pura).

- **Métodos:** atenderUrgencia(mascota, veterinario, dia): Turno (toma el primer horario del veterinario de guardia; si está ocupado, desplaza el turno siguiente y avisa al dueño) y -desplazarEnCascada(turno) (privado: reprograma al primer horario libre y repite si el desplazado pisa a otro).
- **Relaciones:** asociación navegable con Agenda y con Notificador.

6. Inventario y stock

Inventario

Controla el stock de medicamentos, insumos y vacunas, y concentra el descuento al confirmar una consulta. No escribe en ningún log: devuelve las alertas y el controlador las notifica.

- **Atributos:** ninguno propio; los productos son la relación.
- **Métodos:**
    - hayStock(producto, cantidad): boolean: verifica existencias en lotes no vencidos (RN-S2); se usa al indicar un tratamiento, sin descontar.
    - descontarStock(consulta): List<AlertaStock>: al confirmar la consulta, descuenta cada insumo con FEFO, es decir, del lote que vence antes (RN-S3, CU-22). Si el stock total no alcanza, no descuenta nada. **Devuelve las alertas** de stock bajo generadas (sin duplicar una vigente, RN-S1) para que el controlador las notifique.
    - getProductosBajoMinimo(): List<Producto>: base del reporte de stock bajo.
- **Relaciones:** agregación 1 a 0..* con Producto; composición 1 a 0..* con AlertaStock.

Producto, Vacuna, Lote, Proveedor y AlertaStock

- **Producto:** -codigo, -nombre, -stockMinimo, -precioUnitario (S-11). Métodos: getStockActual() (suma de lotes no vencidos, S-16), estaBajoMinimo(), agregarLote(lote), descontar(cantidad) (FEFO) y getDescripcion(). Composición 1 a 0..* con Lote; asociación 0..* a 1 con Proveedor; superclase de Vacuna.
- **Vacuna:** es un producto más (S-25). -especies: Set<Especie>, -edadMinimaSemanas, -intervaloRefuerzoDias. Métodos: aplicaA(especie) (S-13), permiteEdad(semanas) (RN-V3), calcularProximoRefuerzo(fechaAplicacion) y getDescripcion() (sobreescribe el de Producto). VacunaAplicada apunta a ella.
- **Lote:** -numero, -cantidad, -fechaVencimiento (vencimiento físico, independiente del de la inmunidad, S-26). Métodos: estaVencido(fechaActual), descontar(cantidad) y compareTo(otro) (orden por vencimiento, que implementa FEFO). Composición con Producto.
- **Proveedor:** -nombre, -telefono, -correo (S-28). Abastece a 0..* productos; cada producto tiene uno solo.
- **AlertaStock:** -fecha, -stockActual, -stockMinimo, -vigente; método cerrar() cuando se repone el stock. Composición con Inventario; asociación 0..* a 1 con Producto.

7. Facturación y medios de pago

Factura

Comprobante de una consulta confirmada: incluye la consulta y los insumos usados (RN-F1). Una vez emitida no se modifica (RN-F4).

- **Atributos:** -numero, -fecha, -estado: EstadoFactura y -seniaDescontada (parte de la seña del dueño que se descontó, S-04). El medio de pago y el cliente son relaciones, no atributos.
- **Métodos:**
    - agregarLinea(descripcion, cantidad, precioUnitario): **crea** la LineaFactura y la suma al detalle (Creador).
    - calcularSubtotal(): double: suma los subtotales de las líneas.
    - calcularTotal(): double: le pide al Duenio asociado si lo tiene, le pasa al MedioPago el subtotal y ese dato, y resta seniaDescontada (Experto, Polimorfismo).
    - anular(motivo, emisor): NotaCredito: verifica que no esté anulada, pasa a ANULADA sin tocar su contenido y **crea** la NotaCredito por el total (S-20).
    - estaAnulada(): boolean.
- **Relaciones:** composición 1 a 1..* con LineaFactura; asociación con Consulta (0..1 a 1), con Duenio como cliente (0..* a 1) y con MedioPago (0..* a 1); dependencia hacia Tarifario; NotaCredito la anula.

LineaFactura

Un renglón del detalle: la consulta o un insumo.

- **Atributos:** -descripcion, -cantidad, -precioUnitario. **Método:** getSubtotal() (cantidad × precioUnitario).
- **Relaciones:** composición con Factura.

MedioPago (interfaz)

Define cómo se ajusta un subtotal según el medio de pago y el plan de salud (SOLID: O; GRASP: Polimorfismo).

- **Métodos:** calcularTotal(subtotal, tienePlanSalud): double y getNombre(): String.
- **Implementaciones:**
    - **Efectivo:** aplica el mayor entre el 10 % de efectivo y el 15 % del plan; no se acumulan (RN-F2, RN-F3).
    - **Debito:** sin recargo; con plan de salud, 15 % de descuento.
    - **Credito** (-cuotas: int): primero el 15 % del plan, si corresponde, y después el 8 % de recargo sobre ese subtotal (S-19).

NotaCredito

Anula una factura completa; no hay anulaciones parciales (S-20).

- **Atributos:** -numero, -fecha, -motivo, -importe (el total de la factura). **Método:** getImporte().
- **Relaciones:** asociación 0..1 a 1 con Factura; asociación 0..* a 1 con Recepcionista (quien la emite, S-27).

Tarifario

Concentra los valores configurables del negocio (S-03, S-10).

- **Atributos:** -precios: Map<TipoAtencion, Double>, -montoSenia (monto fijo) y -porcCancelacionTardia (20 %).
- **Métodos:** getPrecio(tipo), getMontoSenia(), getPorcCancelacionTardia() y calcularCargoCancelacionTardia(): double (precio de la consulta general por el porcentaje, S-06; es el experto porque tiene ambos datos).
- **Relaciones:** Factura depende de él; ControladorFacturacion lo usa.

8. Servicios, notificaciones y persistencia

Son clases de fabricación pura: no representan conceptos del negocio, existen para que las clases del dominio no mezclen responsabilidades ajenas.

Notificador (interfaz) y NotificadorLog

Simulan el aviso a las personas, por consola o archivo, sin enviar correos reales (S-02). Desacoplan las reglas del negocio de la forma de notificar (SOLID: D).

- **Notificador:** notificar(destinatario: Persona, mensaje: String). El destinatario es una Persona, así sirve tanto para el dueño como para el administrador.
- **NotificadorLog:** -rutaLog: String; implementa notificar escribiendo el mensaje en el log.
- **Relaciones:** NotificadorLog implementa Notificador; lo usan ControladorTurnos, ControladorAtencion, GestorUrgencias y ServicioRecordatorios.

ServicioRecordatorios

Emite los recordatorios de vacunas próximas a vencer (RN-V2, CU-20).

- **Métodos:** emitirRecordatorios(mascotas, fechaActual): recorre las vacunas aplicadas de cada mascota y, por cada una que cumple requiereRecordatorio(fechaActual), notifica al dueño.
- **Relaciones:** asociación navegable con Notificador.

GeneradorReportes

Concentra los reportes del administrador (CU-31, RF-19).

- **Métodos:** turnosPorVeterinario(veterinario, desde, hasta), productosConStockBajo() y lotesProximosAVencer(fechaActual, diasAnticipacion): List<Lote>, que devuelve los lotes de cualquier producto (incluidas las vacunas) cuyo vencimiento físico cae dentro de los días de anticipación indicados.
- **Relaciones:** asociación navegable con Agenda (para los turnos) y con Inventario (para el stock bajo y los lotes por vencer).

Repositorio<T> (interfaz) y RepositorioArchivo<T>

Guardan y recuperan los datos en archivos, para que se conserven entre ejecuciones.

- **Repositorio<T> (interfaz):** define guardar(lista: List<T>) y cargar(): List<T>. Los controladores dependen de esta interfaz y no de la forma concreta de guardar.
- **RepositorioArchivo<T>:** implementa Repositorio<T>. Tiene -rutaArchivo: String, guarda y lee la lista en un archivo y concentra el manejo de archivos y de sus excepciones (try, catch, finally). Al ser genérico, se instancia con el tipo que se quiere persistir; por ejemplo, RepositorioArchivo<Duenio> guarda a los dueños con sus mascotas e historia clínica.
- **Relaciones:** RepositorioArchivo implementa Repositorio. Se asocian con la interfaz ControladorRegistro, ControladorTurnos, ControladorFacturacion y ControladorAdministracion.

9. Enumeraciones

Enumeración	Valores	Se usa en
TipoAtencion	CONSULTA_GENERAL(30), VACUNACION(15), CIRUGIA(120); entre paréntesis, la duración en minutos	Turno, Tarifario
EstadoTurno	PROGRAMADO, ATENDIDO, CANCELADO, AUSENTE	Turno
EstadoConsulta	EN_CURSO, CONFIRMADA	Consulta
EstadoFactura	EMITIDA, ANULADA	Factura
Especie	PERRO, GATO, AVE, CONEJO	Mascota.getEspecie(), Vacuna
Sexo	MACHO, HEMBRA	Mascota
TamanioPerro	PEQUENO, MEDIANO, GRANDE	Perro

10. Jerarquías y patrones aplicados

Jerarquías con sobreescritura:

- Persona → Duenio, Veterinario, Recepcionista, Administrador (getRol()).
- Mascota → Perro, Gato, Ave, Conejo (getEspecie(), getDatosPropios()).
- Producto → Vacuna (getDescripcion()).
- MedioPago → Efectivo, Debito, Credito (calcularTotal, getNombre).
- Notificador → NotificadorLog y Repositorio → RepositorioArchivo.

Comparable lo implementan Turno (fecha y hora) y Lote (vencimiento, para FEFO).

Principios y patrones, con el problema que resuelven:

Principio	Problema detectado	Decisión	Clases
SOLID: Responsabilidad única	Agenda reunía asignación, cancelación, urgencias y notificación; los reportes estaban repartidos	Se separan las urgencias y los reportes	GestorUrgencias, GeneradorReportes, Agenda
SOLID: Abierto/cerrado	Factura.calcularTotal necesitaba condicionales por medio de pago	Interfaz con una clase por medio de pago	MedioPago, Efectivo, Debito, Credito, Factura
SOLID: Inversión de dependencias	Agenda e Inventario escribían directamente en un log (cada una con su rutaLog)	Se depende de la interfaz Notificador	Notificador, NotificadorLog, controladores y servicios
GRASP: Controlador	Los casos de uso no tenían punto de entrada	Cinco controladores por familia de casos de uso	Controlador*
GRASP: Experto	Recepcionista.emitirNotaCredito usaba el total de una factura ajena; Factura.calcularTotal(planSalud) no conocía a su cliente	Factura.anular crea la nota; Factura se asocia con Duenio	Factura, NotaCredito, Duenio, Recepcionista
GRASP: Creador	Las partes de las composiciones llegaban ya creadas desde afuera	El todo crea sus partes	HistoriaClinica, Consulta, Factura
GRASP: Fabricación pura	Urgencias, notificación, reportes y archivos estaban dentro de clases del dominio	Clases de servicio sin concepto de negocio	GestorUrgencias, GeneradorReportes, ServicioRecordatorios, NotificadorLog, RepositorioArchivo